

第22回 実践：Web ゲームプログラミング1（ゲームループ）

- 前回：Canvas に自機を描いた
- 今回：自機を自動で動かし続ける（ゲームループ）
- 次回：キーボードで自機を動かす

実習時のフォルダ構成

Web_名前 の中に 22 フォルダを作る。22 の中に sample と exercise を作り、次のファイルを用意する。

```
Web_名前
├── 22
│   ├── sample
│   │   ├── index.html
│   │   ├── style.css
│   │   └── main.js
│   └── exercise
│       ├── index.html
│       ├── style.css
│       └── main.js
```

- sample：講義で使うサンプル。今回から第 28 回まで、この sample に少しずつ書き足して、シューティングゲームを作っていく。
- exercise：自分のシューティングゲーム。第 21 回の exercise で作った自機（player と drawPlayer）をもとに作り始め、講義と同じ機能を、自分のゲームにも追加していく。

sample/index.html と exercise/index.html には、どちらも次の内容を書く。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>シューティングゲーム</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <canvas id="game" width="400" height="500"></canvas>
  <script src="main.js"></script>
</body>
</html>
```

sample/style.css と exercise/style.css には、どちらも次の内容を書く。

```
body {
  margin: 0;
  overflow: hidden;
}
```

- `style.css` の CSS は、ゲームを遊びやすくするための設定。CSS は、第 16 回と同じく、HTML とは別のファイル（`style.css`）に書き、`<link>` で読み込む。
- `margin: 0;` : ページのまわりの余白をなくし、キャンバスを左上にぴったり置く。
- `overflow: hidden;` : ページがスクロールしないようにする。これがないと、ゲーム中にスペースキーや矢印キーを押したとき、ページが上下に動いてしまうことがある。
- ページがスクロールしないので、ブラウザのウィンドウがキャンバス（幅 400・高さ 500）より小さいと、はみ出した部分が見えなくなる。ウィンドウを大きくして使う。

講義内容

`sample/main.js` に、上から順に書く。書いたら `sample/index.html` をブラウザで開き、画面で結果を確認する。

1. 前回の復習

- `const player = { x: 184, y: 440, w: 32, h: 32, speed: 5 };` のように、自機の位置と大きさをオブジェクトにまとめた。
- `drawPlayer()` で、`player` の位置に四角形を描いた。

前回は、絵を 1 回描くだけだった。今回は、絵を何回も描き直して、自機を動かす。今回の `sample/main.js` は、第 21 回の「Canvas を使う準備の 2 行」「`player`」「`drawPlayer`」をもとにして作る。

2. アニメーションの仕組み

パラパラ漫画のように、「少しだけ動かす → 描き直す」を素早く繰り返すと、動いて見える。

少し動かす（更新） → 描く（描画） → 少し動かす → 描く → …

- この「更新 → 描画」の 1 回分を、**1 フレーム**という。
- ブラウザでは、1 秒間に約 60 フレーム（60 回）繰り返す。
- 正確には、画面（モニター）が 1 秒間に表示を書き換える回数と同じだけ繰り返す。多くの画面は 60 回だが、120 回や 144 回の画面では、ゲームが 2 倍以上の速さで動く。この授業では、60 回として考える。
- この繰り返しを「ゲームループ」という。

3. 今回の main.js の並び

```
// ===== 準備 =====      ← 第 21 回と同じ 2 行
// ===== ゲームの状態 =====
// ===== 自機 =====      ← 第 21 回の player
// ===== 更新 =====      ← 今回追加する
// ===== 描画 =====      ← 第 21 回の drawPlayer と、今回追加する関数
// ===== ゲームループ ===== ← 今回追加する
```

- `// ===== 準備 =====` のような見出しのコメントで、ファイルの中を役割ごとに分けておく。これから書き足す場所が分かりやすくなる。

4. 準備と変数

```
// ===== 準備 =====
const canvas = document.getElementById("game");
const ctx = canvas.getContext("2d");

// ===== ゲームの状態 =====
let frameCount = 0;           // ゲーム開始からのフレーム数

// ===== 自機 =====
const player = { x: 184, y: 440, w: 32, h: 32, speed: 5 };
```

- `frameCount` : 何フレーム目かを数える変数。1 フレームごとに 1 増やす。
- `player` は第 21 回と同じ。 `speed` (速さ) を、今回から使う。

5. 更新（動かす）と、const のオブジェクト

```
// ===== 更新 =====
function updatePlayer() {
  player.x += player.speed;    // const の player でも、中の値は書き換えられる
  // 右端から出たら、左端に戻す
  if (player.x > canvas.width) {
    player.x = -player.w;
  }
}

function update() {
  frameCount += 1;
  updatePlayer();
}
```

- `updatePlayer` : 自機を、1 フレームで `speed` (5) だけ右へ動かす。1 秒では約 300 (5 × 60) 動く。
- 右端から完全に出たら (`player.x` が `canvas.width` より大きくなったら)、左端の外 (`-player.w`) に戻す。 `-player.w` は `0 - player.w` と同じ意味で、ここでは `-32` になる。自機の右端が、ちょうど画面の左端にくる位置。
- `update` : 1 フレームで行う「更新」を、まとめて呼び出す関数。

constなのに、なぜ player.x を変えられるのか

第 21 回では `player.x = 100;` と書き、今回は `player.x += player.speed;` と書いている。player は `const` で作ったのに、中の値を変えられる。

- `const` で禁止されるのは、「変数そのものに、別の値を代入し直すこと」（例：`player = { x: 0, y: 0 };` はエラーになる）。
- 変数の中のオブジェクトはそのまま、そのプロパティの値を書き換えること（`player.x = 100;`）は、`const` でもできる。配列の要素（`enemyHps[1] = 20;`）も同じ。
- この授業では、ここから次のルールにする。
- 変数そのものに、別の値を代入し直すことがある → `let`
- 代入し直さない（中の値を書き換えるだけの場合も含む） → `const`
- `player` は、プロパティを書き換えるだけで、変数そのものは代入し直さないので `const`。
`frameCount` は、`frameCount += 1` で変数そのものの値が変わるので `let`。

6. 描画（描く）

```
// ===== 描画 =====
function drawBackground() {
  ctx.fillStyle = "#000022";
  ctx.fillRect(0, 0, canvas.width, canvas.height);
}

function drawPlayer() {
  ctx.fillStyle = "#00ccff";
  ctx.fillRect(player.x, player.y, player.w, player.h);
}

function draw() {
  drawBackground();
  drawPlayer();
}
```

- `drawPlayer` は第 21 回と同じ。
- `draw` : 1 フレームで行う「描画」を、まとめて呼び出す関数。背景 → 自機の順に描く。
- 毎フレーム、最初に背景でキャンバス全体を塗りつぶして、前のフレームの絵を消してから描き直す。

7. ゲームループ

```
// ===== ゲームループ =====
function gameLoop() {
  update();
  draw();
  requestAnimationFrame(gameLoop);
}

gameLoop();
```

- `requestAnimationFrame(gameLoop);` : 「次の画面の書き換えのタイミングで、`gameLoop` を呼び出してください」とブラウザに頼む。
- `gameLoop` の最後で毎回これを頼むので、`gameLoop` が約 60 回／秒で繰り返し呼び出される。
- `()` の中には、関数の名前だけを書く (`gameLoop()` のように `()` を付けない)。第 20 回の `scoreButton.addEventListener("click", addTenPoints);` と同じで、関数そのものをブラウザに渡している。
- 最後の `gameLoop();` で、最初の 1 回を呼び出して、ゲームループを始める。
- `F12` のコンソールに `frameCount` と入力して `Enter` キーを押すと、その時点のフレーム数が分かる。何回か入力して、どんどん増えていることを確かめる。
- 背景を描かないとどうなるかを確かめる：`draw` 中の `drawBackground();` の行の先頭に `//` を付けると、前のフレームの絵が消えずに残り、自機が線のように伸びて見える。確認したら、`//` を消して元に戻す。

8. うまく動かないとき

症状	確認すること
何も表示されない	コンソールの赤字のエラー。 <code>getElementById("game")</code> と <code><canvas id="game"></code> の名前
自機が止まったまま	最後の <code>gameLoop();</code> を書いたか。 <code>requestAnimationFrame(gameLoop)</code> の綴り
自機が 1 回しか動かない	<code>gameLoop</code> の中に <code>requestAnimationFrame(gameLoop);</code> があるか
自機が線のように伸びる	<code>draw</code> の最初で <code>drawBackground();</code> を呼んでいるか

演習

`exercise/main.js` (自分のシューティングゲーム) に書く。結果は、`exercise/index.html` をブラウザで開いて確認する。

演習 1 ゲームループを作る

- 講義と同じ形で、準備・自機・更新・描画・ゲームループを書き、自機が右へ動き続けるようにする。
- 自機は、第 21 回の `exercise` で作った `player` と `drawPlayer` (自分で決めた色・大きさ) を使う。`player` に `speed` がなければ加える。
- 右端から出たら、左端に戻るようにする。

演習 2 速さを変えて確かめる

- `player` の `speed` を 10 にすると、1 秒で約何 px 動くかを予測してから (1 秒は約 60 フレーム)、実際に動かして確かめる。

- `speed` を 2 などにも変えて、動きがどう変わるかを確かめる。最後に、自分がよいと思う値にする。
- コンソールに `frameCount` と入力して、フレーム数が増えていることを確かめる。

発展（任意）

- 30 フレームごとに、自機の色が 2 つの色で交互に切り替わるようにする。（ヒント：
`Math.floor(frameCount / 30) % 2` が 0 か 1 かで、`fillStyle` を変える。）

この回の終わりの `sample/main.js` 全体

途中で分からなくなったら、次と見比べる。

```

// ===== 準備 =====
const canvas = document.getElementById("game");
const ctx = canvas.getContext("2d");

// ===== ゲームの状態 =====
let frameCount = 0;           // ゲーム開始からのフレーム数

// ===== 自機 =====
const player = { x: 184, y: 440, w: 32, h: 32, speed: 5 };

// ===== 更新 =====
function updatePlayer() {
    player.x += player.speed;    // const の player でも、中の値は書き換えられる
    // 右端から出たら、左端に戻す
    if (player.x > canvas.width) {
        player.x = -player.w;
    }
}

function update() {
    frameCount += 1;
    updatePlayer();
}

// ===== 描画 =====
function drawBackground() {
    ctx.fillStyle = "#000022";
    ctx.fillRect(0, 0, canvas.width, canvas.height);
}

function drawPlayer() {
    ctx.fillStyle = "#00ccff";
    ctx.fillRect(player.x, player.y, player.w, player.h);
}

function draw() {
    drawBackground();
    drawPlayer();
}

// ===== ゲームループ =====
function gameLoop() {
    update();
    draw();
    requestAnimationFrame(gameLoop);
}

gameLoop();

```